SKYHACK3.0

Proactive Flight Complexity Management: A Data-Driven Framework for Optimizing Ground Operations

Team - Query Kings
Punjab Engineering College

Table of content

Executive Summary

Problem
- Flight difficulty at ORD is currently assessed manually → inconsistent prioritization and reactive resourcing.

Solution
- Built a daily Flight Difficulty Score for every departure.
- Flights ranked and bucketed into Difficult / Medium / Easy → enables proactive operations.

Data & Build
- 5 datasets integrated with DuckDB SQL → Master table: 8,063 flights × 27 features.
- Data quality: no duplicates; negligible nulls dropped; airport codes standardized; timestamps parsed.
- Totals: 279,049 checked bags, 353,007 transfer bags, 17,065 SSR requests.

Modeling & Score
- Target = Difficult if departure delay > 15 min (27.8% of flights).
- Logistic Regression (baseline): AUC ~0.69; confirmed key drivers.
- Random Forest (chosen): higher accuracy; generates Difficulty Score (0–1).
- Daily reset: scores ranked within each date (1 = hardest).
- Buckets: Difficult (top 20%), Medium (20–50%), Easy (50–100%).

Key Drivers of Difficulty
- Low slack minutes (scheduled ground < minimum turn).
- High SSR loads (wheelchairs, strollers, children).
- Transfer baggage share & volume (esp. hot transfers <30 min).
- International destinations show consistently higher difficulty.

Delay Reality by Class (positive delay minutes)
- Easy: ~3.1 min average
- Medium: ~22.9 min average
- Difficult: ~74.7 min average

Executive Summary

Control-Room View (Heatmap Example)

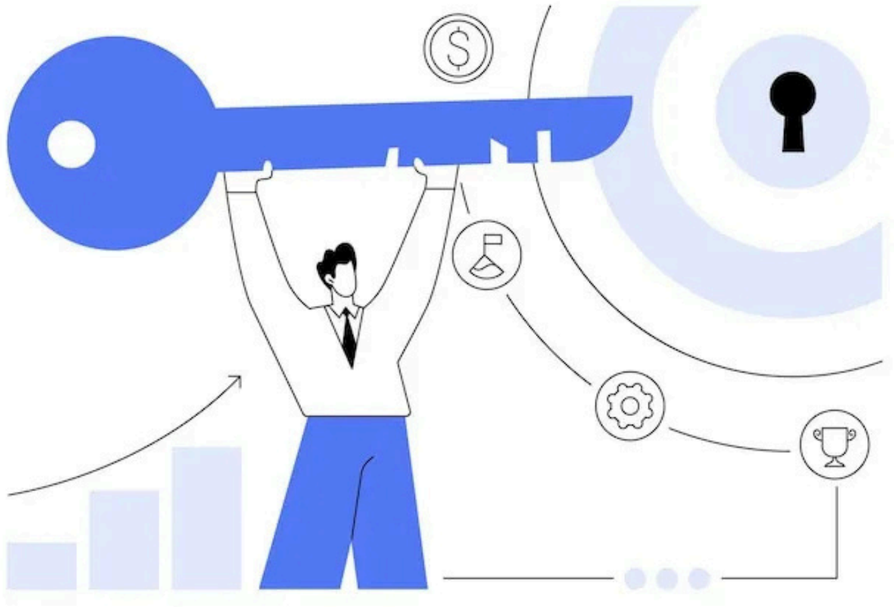
- On busiest day (Aug 12) → $2.75M in departure delay cost (@ $75/min).
- High-risk flights (red clusters) → mostly international with high SSR and transfer baggage.
- Enables control-room managers to pre-stage ramp & crew before issues occur.

Economic Impact (if Difficult flights trimmed)

- 3 min each: 4,607 min/day saved → $345K/day (~$10M/month).
- 5 min each: 7,641 min/day saved → $573K/day (~$17M/month).
- 10 min each: 15,105 min/day saved → $1.13M/day (~$34M/month).

Recommendations

- Pre-allocate resources for Difficult flights at morning huddles.
- SSR-heavy flights: assign dedicated staff; pilot pre-boarding flows.
- Transfer-heavy flights: fast-track hot transfer baggage.
- Low-slack repeat flights: schedule +20–30 min buffer for sustainable improvement.



Problem Statement

The Challenge

- Frontline teams ensure every flight departs on time, yet operational complexity varies significantly across flights.
- Contributing factors include limited ground time, high baggage volumes, and greater service needs with larger passenger loads.

The Gap

- Flight difficulty is currently assessed through staff experience and local knowledge.
- This approach is inconsistent, non-scalable, and limits proactive resource allocation.

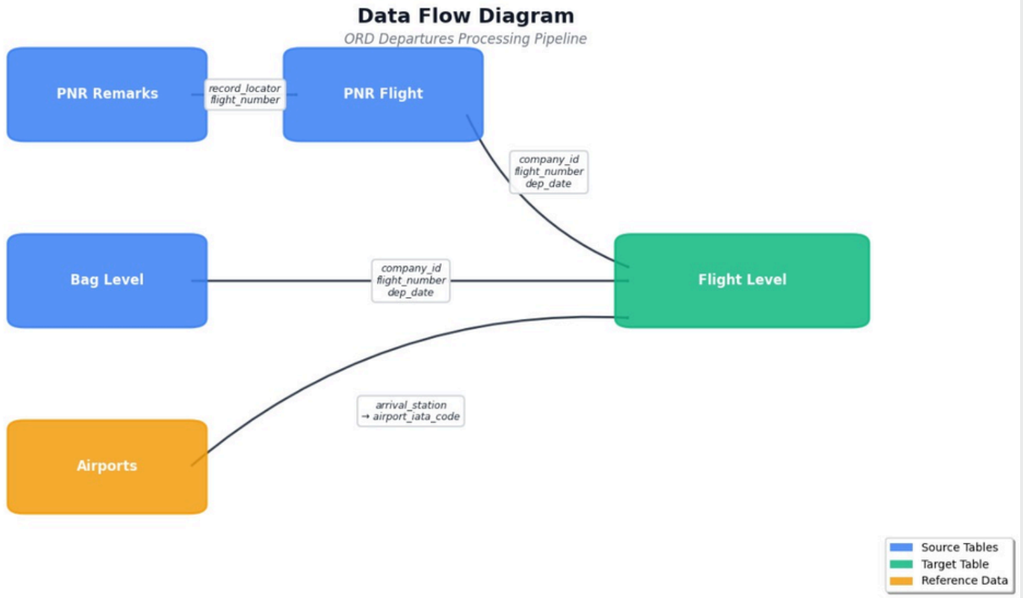
The Opportunity

- A standardized Flight Difficulty Score provides a data-driven method to quantify operational complexity.
- This enables consistent decision-making, improved planning, and more efficient deployment of resources across the airport.

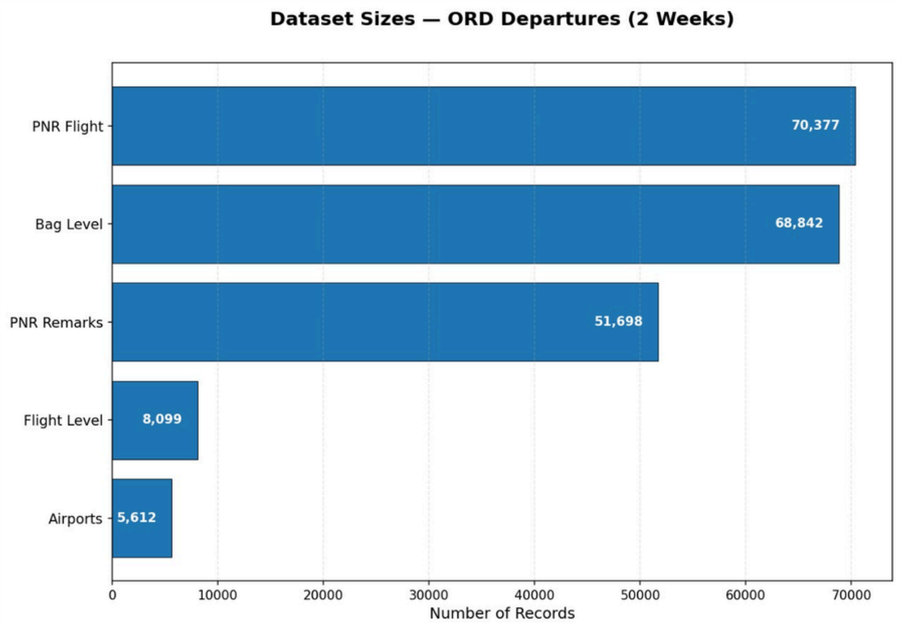
Understanding the data

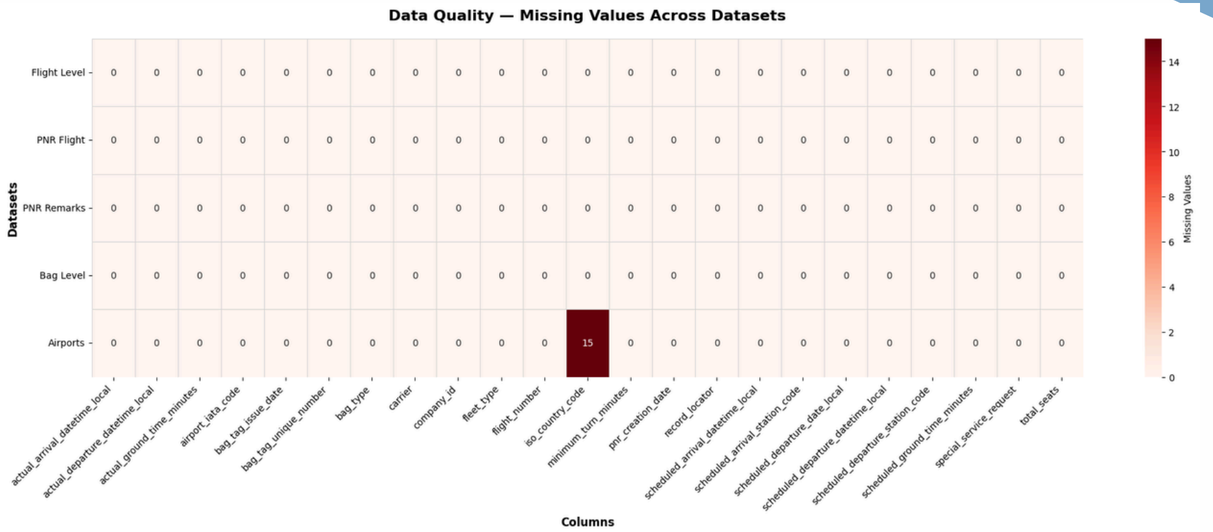
Schema & Relationships

- Flight Level Table (center, big box)
  - Key columns: company_id, flight_number, scheduled_departure_date_l
  - This is the hub.
- PNR Flight Table (left)
  - Linked by: (company_id, flight_number, scheduled_departure_date_loc
  - Adds: passenger info (pax, lap child, basic econ, stroller).
- PNR Remarks Table (below PNR Flight)
  - Linked by: record_locator, flight_number
  - Adds: Special Service Requests (SSR).
- Bag Level Table (right)
  - Linked by: (company_id, flight_number, scheduled_departure_date_loc
  - Adds: baggage info (checked, transfer, hot transfer).
- Airport Table (top)
  - Linked by: scheduled_arrival_station_code → airport_iata_code
  - Adds: country / region info.



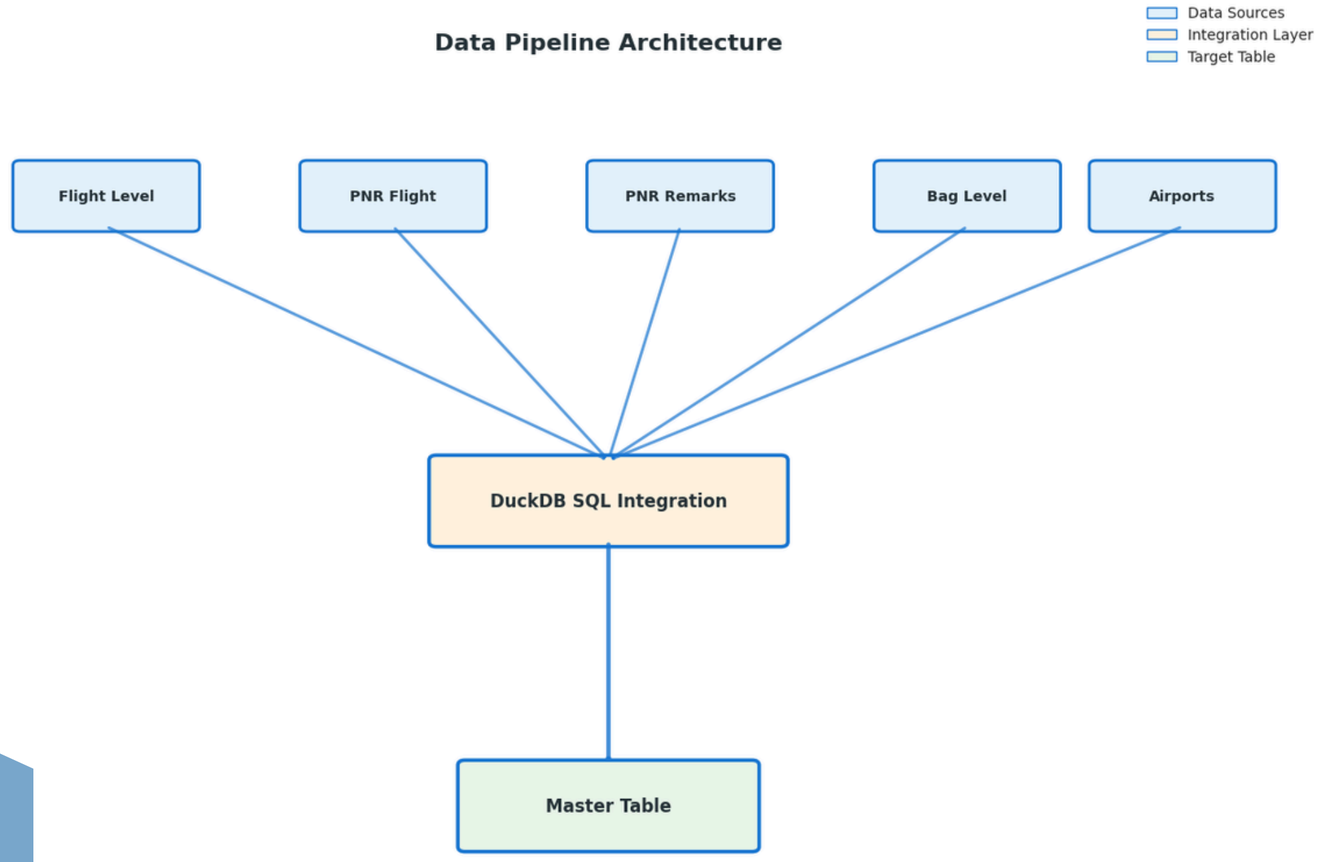
Quality and Cleaning





- Data Quality: Minimal missing values (<0.05%, safely dropped) and no duplicates across 5 datasets.
- Standardization: Dates converted to timestamps; codes normalized to uppercase.

Building the Master Table with SQL & DuckDB



Why DuckDB?

- In-memory SQL engine, fast joins, works directly in Colab.

Process

- Used company_id + flight_number + date as keys.
- Joined PNR for passenger info, Remarks for SSR, Bags for baggage, Airport for country.

Output

- Final Master Table: 8,099 flights × 27 features.
- Ready for EDA, feature engineering, and ML.

Building the Master Table with SQL & DuckDB

```
con.execute("""
CREATE OR REPLACE VIEW v_flights_ord AS
SELECT *
FROM (
  SELECT
    *,
    ROW_NUMBER() OVER (
      PARTITION BY company_id, flight_number, scheduled_departure_date_local,
                   scheduled_departure_station_code, scheduled_arrival_station_code
      ORDER BY COALESCE(actual_departure_datetime_local, scheduled_departure_datetime_local) DESC
    ) AS rn
  FROM flights
  WHERE scheduled_departure_station_code = 'ORD'
) t
WHERE rn = 1
""")
n_flights = con.execute("SELECT COUNT(*) FROM v_flights_ord").fetchone()[0]
```

```
import duckdb, pandas as pd, numpy as np
con = duckdb.connect()


con.register("flights", flights)
con.register("pnr_flight", pnr_flight)
con.register("pnr_rem", pnr_rem)
con.register("bag_level", bag_level)
con.register("airports", airports)
```

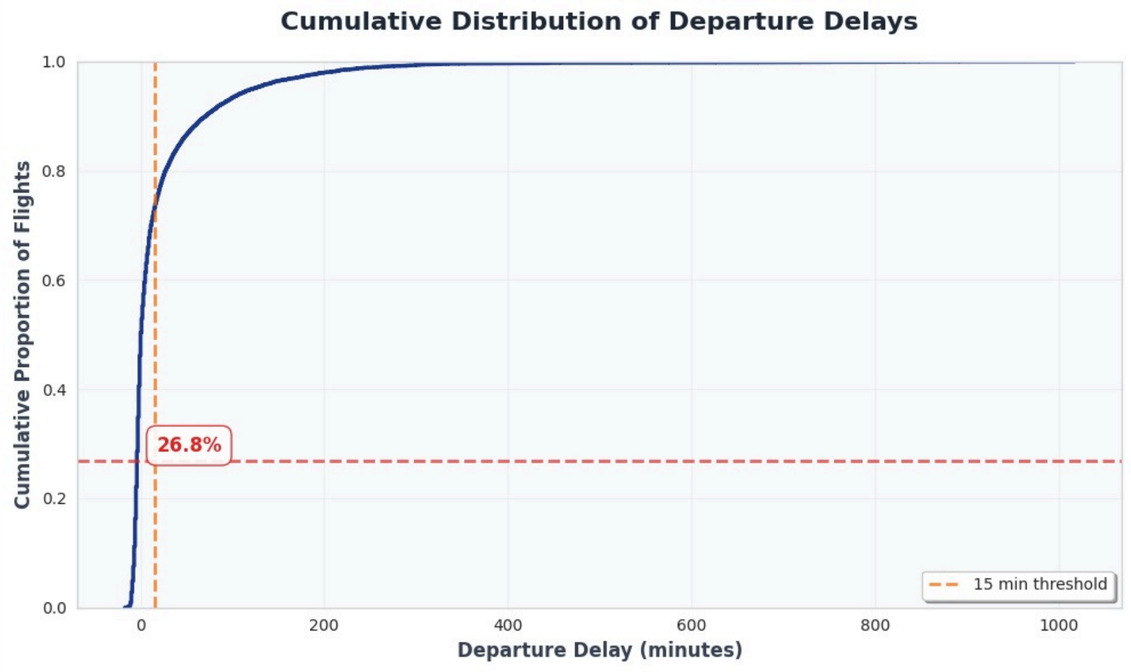
```
df_master = con.execute("""
WITH base AS (
  SELECT
    f.*,
    (f.scheduled_ground_time_minutes - f.minimum_turn_minutes) AS slack_mins,
    EXTRACT(hour FROM CAST(f.scheduled_departure_datetime_local AS TIMESTAMP)) AS dep_hour,
    EXTRACT(dow  FROM CAST(f.scheduled_departure_datetime_local AS TIMESTAMP)) AS dep_dow
  FROM v_flights_ord f
)
SELECT
  b.*,
  COALESCE(p.pax_total,0)        AS pax_total,
  COALESCE(p.lap_childs,0)       AS lap_childs,
  COALESCE(p.basic_econ,0)       AS basic_econ,
  COALESCE(p.stroller_users,0)   AS stroller_users,
  COALESCE(p.distinct_pnrs,0)    AS distinct_pnrs,
  COALESCE(bg.checked_bags,0)    AS checked_bags,
  COALESCE(bg.transfer_bags,0)   AS transfer_bags,
  CASE WHEN COALESCE(bg.checked_bags,0) > 0
       THEN CAST(bg.transfer_bags AS DOUBLE)/bg.checked_bags
       ELSE NULL END             AS transfer_ratio,
  COALESCE(s.ssr_count,0)        AS ssr_count,
  apt.iso_country_code           AS arrival_country
FROM base b
LEFT JOIN v_pnr_agg  p  USING (company_id, flight_number, scheduled_departure_date_local,
                               scheduled_departure_station_code, scheduled_arrival_station_code)
LEFT JOIN v_bags_agg bg USING (company_id, flight_number, scheduled_departure_date_local,
                               scheduled_departure_station_code, scheduled_arrival_station_code)
LEFT JOIN v_ssr_agg  s  USING (company_id, flight_number, scheduled_departure_date_local,
                               scheduled_departure_station_code, scheduled_arrival_station_code)
LEFT JOIN airports apt
       ON b.scheduled_arrival_station_code = apt.airport_iata_code
""").df()

print("Master rows:", len(df_master))
assert len(df_master) == n_flights, "Row-count mismatch: master != flights ORD"
```

- Integrated 5 raw datasets into 1 Master Table using DuckDB SQL joins.
- Ensured no duplication and handled minimal missing values.
- Result: a clean, unified dataset (8K flights, 27 features) → foundation for EDA & ML.

Deliverable 1

1.1 Exploratory Data Analysis (EDA)

Average Delay and Percentage of Late Departures

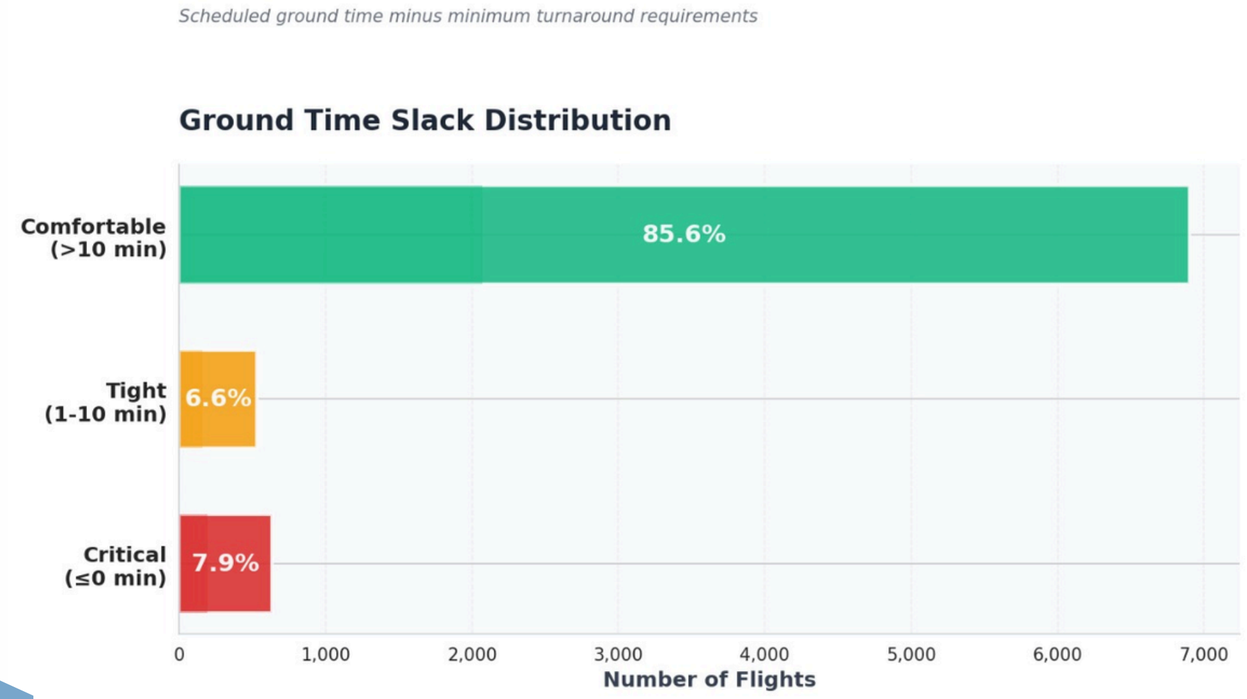



```
plt.figure(figsize=(9,6))
sns.ecdfplot(df_master["dep_delay_min"], color="navy")

# 15-min cutoff
plt.axvline(15, color="orange", linestyle="--", label="15 min threshold")
plt.axhline(late_flights_pct/100, color="red", linestyle="--", alpha=0.7)
plt.text(15+1, late_flights_pct/100, f"{late_flights_pct:.1f}%",
         color="red", fontsize=11, va="bottom", weight="bold")

plt.title("ECDF of Departure Delays", fontsize=14, weight="bold")
plt.xlabel("Departure Delay (minutes)")
plt.ylabel("Cumulative Proportion of Flights")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

Average Departure Delay: 21.22 minutes
Percentage of Flights Delayed >15 min: 26.78%

1.2 Exploratory Data Analysis (EDA)

Number of flights with scheduled ground time at or below the minimum turn threshold.




```python
# Slack minutes = scheduled ground time - minimum turnaround
df_master["slack_mins"] = df_master["scheduled_ground_time_minutes"] - df_master["minimum_turn_minutes"]

# Bucket flights by slack (clean, explicit bins)
bins = [-np.inf, 0, 10, np.inf]
labels = ["Critical (≤0 min)", "Tight (1-10 min)", "Comfortable (>10 min)"]
cats = pd.cut(df_master["slack_mins"], bins=bins, labels=labels, right=True)

# Counts and percentages
counts = cats.value_counts().reindex(labels, fill_value=0)
total = int(counts.sum())
pct = (counts / total * 100).round(1)

print(f"Flights at/below minimum (≤0): {counts[labels[0]]} ({pct[labels[0]]}%)")
print(f"Flights with tight slack (1–10): {counts[labels[1]]} ({pct[labels[1]]}%)")
print(f"Flights comfortable (>10): {counts[labels[2]]} ({pct[labels[2]]}%)")
print("Total flights:", total)

# Plot: simple, readable defaults (no gradients, no heavy styling)
fig, ax = plt.subplots(figsize=(9, 5), dpi=150)

bars = ax.barh(counts.index, counts.values, height=0.6)

# Percent labels centered in bars
ax.bar_label(bars, labels=[f"{v:.1f}%" for v in pct.values], label_type="center",
             fontsize=11, weight="bold", color="white")

# Axes/text
ax.set_title("Ground Time Slack Distribution", fontsize=14, loc="left")
ax.set_xlabel("Number of flights", fontsize=11)

# Light grid and thousands on x-axis
ax.grid(axis="x", linestyle="--", alpha=0.3)
ax.xaxis.set_major_formatter(FuncFormatter(lambda x, pos: f"{int(x):,}"))

# Keep spines minimal
for spine in ("top", "right"):
    ax.spines[spine].set_visible(False)

plt.tight_layout()
plt.show()
```

Flights at/below minimum (≤0): 633 (7.9%)

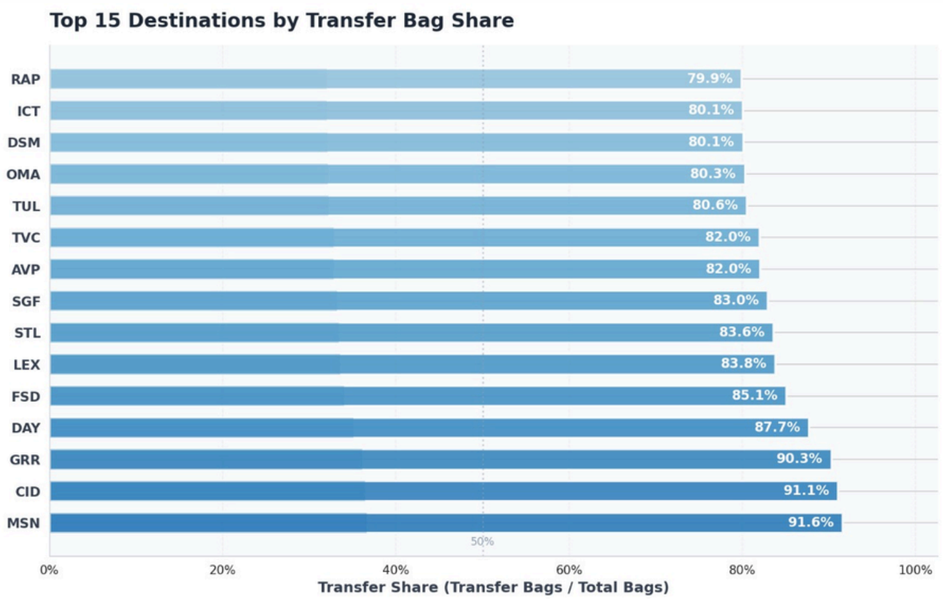
Flights with tight slack (1–10): 530 (6.6%)

Flights comfortable (>10): 6900 (85.6%)

Total flights: 8063

1.3 Exploratory Data Analysis (EDA)

Average Transfer-to-Checked Bag Ratio



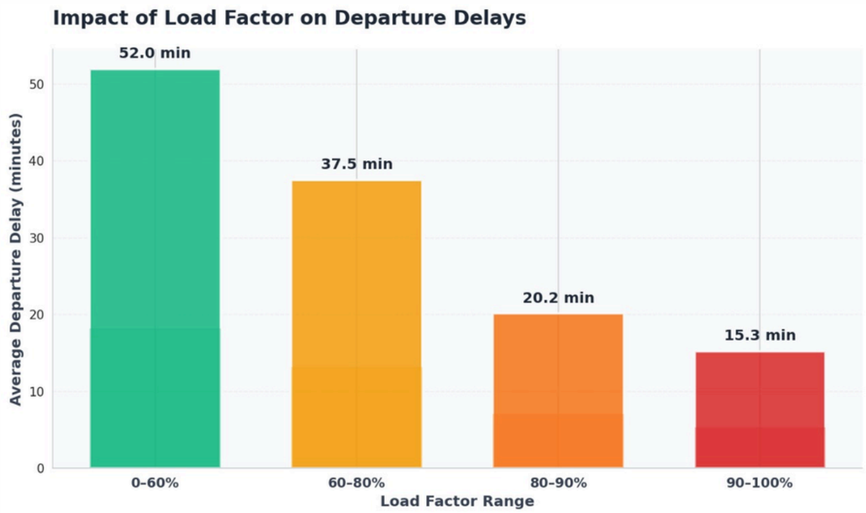

```
    valid.groupby("scheduled_arrival_station_code")[["transfer_bags", "checked_bags"]]
         .sum()
         .query("checked_bags > 0")
         .assign(transfer_share=lambda d: d["transfer_bags"] / (d["transfer_bags"] + d["checked_bags"]))
)

top15 = by_dest.sort_values("transfer_share", ascending=False).head(15)

fig, ax = plt.subplots(figsize=(10, 6), dpi=150)

y_pos = np.arange(len(top15))
x_vals = top15["transfer_share"].values
destinations = top15.index

bars = ax.barh(y_pos, x_vals, color="#3b82f6", alpha=0.85)

for i, v in enumerate(x_vals):
    ax.text(v + 0.01, i, f"{v:.1%}", va="center", ha="left", fontsize=10, weight="bold")

ax.set_yticks(y_pos)
ax.set_yticklabels(destinations, fontsize=10)
ax.set_xlabel("Transfer Share", fontsize=11)
ax.set_title("Top 15 Destinations by Transfer Bag Share", fontsize=14, loc="left")

ax.xaxis.set_major_formatter(plt.FuncFormatter(lambda x, p: f"{x:.0%}"))
ax.set_xlim(0, max(x_vals) * 1.15)

ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.grid(axis="x", linestyle="--", alpha=0.3)

plt.tight_layout()
```

```
valid = df_master[df_master["checked_bags"] > 0].copy()

valid["transfer_ratio"] = valid["transfer_bags"] / valid["checked_bags"]
valid["transfer_share"] = valid["transfer_bags"] / (valid["transfer_bags"] + valid["checked_bags"])

unweighted_mean_ratio = valid["transfer_ratio"].mean()
weighted_mean_ratio = valid["transfer_bags"].sum() / valid["checked_bags"].sum()

unweighted_mean_share = valid["transfer_share"].mean()
weighted_mean_share = valid["transfer_bags"].sum() / (valid["transfer_bags"].sum() + valid["checked_bags"].sum())

print(f"Unweighted mean ratio (transfer/checked): {unweighted_mean_ratio:.3f}")
print(f"Weighted mean ratio   (transfer/checked): {weighted_mean_ratio:.3f}")
print(f"Unweighted mean share (transfer/total):   {unweighted_mean_share:.3f}")
print(f"Weighted mean share   (transfer/total):   {weighted_mean_share:.3f}")
```

- MSN, CID and GRR record the highest transfer-to-checked bag ratios.
- These airports primarily function as layover hubs within the network.

1.4 Exploratory Data Analysis (EDA)

Analyze passenger load variations across flights and their correlation with operational difficulty.



Flights at/below minimum (≤0): 633 (7.9%)

Flights with tight slack (1–10): 530 (6.6%)

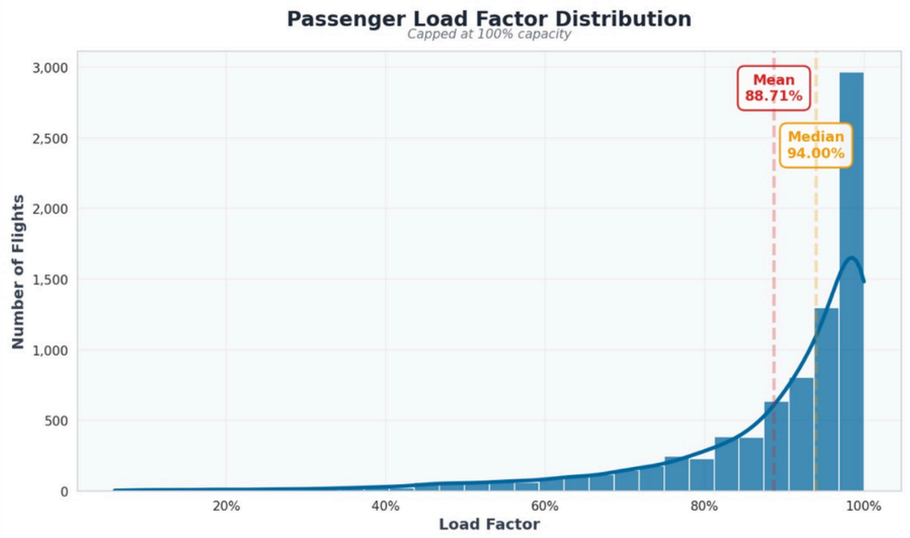
Flights comfortable (>10): 6900 (85.6%)

Total flights: 8063

```
lf = (df_master["pax_total"] / df_master["total_seats"]).clip(lower=0, upper=1)
df_master["load_factor"] = lf

fig, ax = plt.subplots(figsize=(8, 5), dpi=150)
ax.hist(lf, bins=30)

mean_lf = float(lf.mean())
median_lf = float(lf.median())

ax.axvline(mean_lf, linestyle="--", linewidth=1)
ax.axvline(median_lf, linestyle=":", linewidth=1)

ax.set_title("Load Factor Distribution")
ax.set_xlabel("Load Factor")
ax.set_ylabel("Flights")
ax.xaxis.set_major_formatter(FuncFormatter(lambda x, p: f"{x:.0%}"))

ax.text(mean_lf, ax.get_ylim()[1]*0.95, f"Mean {mean_lf:.2%}", ha="left", va="top")
ax.text(median_lf, ax.get_ylim()[1]*0.88, f"Median {median_lf:.2%}", ha="left", va="top")

plt.tight_layout()
plt.show()

df_master["load_bin"] = pd.cut(
    df_master["load_factor"],
    bins=[0, 0.6, 0.8, 0.9, 1.0],
    labels=["0-60%", "60-80%", "80-90%", "90-100%"],
    include_lowest=True,
    right=True
)

delay_by_load = (
    df_master.groupby("load_bin", observed=True)["dep_delay_min"]
    .mean()
    .reindex(["0-60%", "60-80%", "80-90%", "90-100%"])
)

fig, ax = plt.subplots(figsize=(8, 5), dpi=150)
bars = ax.bar(range(len(delay_by_load)), delay_by_load.values)

ax.set_title("Average Departure Delay by Load Factor")
ax.set_xlabel("Load Factor Range")
ax.set_ylabel("Average Delay (min)")
ax.set_xticks(range(len(delay_by_load)))
ax.set_xticklabels(delay_by_load.index)

for i, v in enumerate(delay_by_load.values):
    ax.text(i, v, f"{v:.1f}", ha="center", va="bottom")

plt.tight_layout()
plt.show()
```

1.4 Exploratory Data Analysis (EDA)

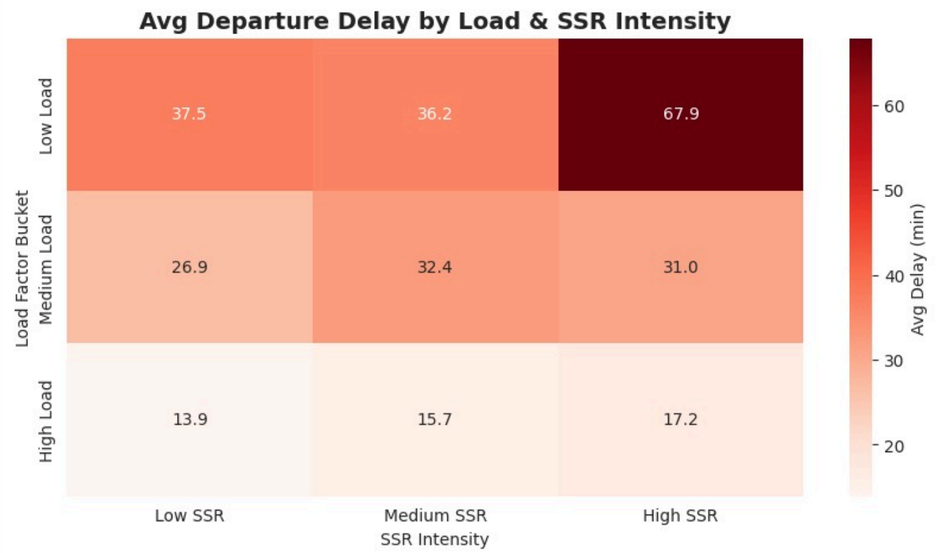
Analyze passenger load variations across flights and their correlation with operational difficulty.




```python
# Slack minutes = scheduled ground time - minimum turnaround
df_master["slack_mins"] = df_master["scheduled_ground_time_minutes"] - df_master["minimum_turn_minutes"]

# Bucket flights by slack (clean, explicit bins)
bins = [-np.inf, 0, 10, np.inf]
labels = ["Critical (≤0 min)", "Tight (1–10 min)", "Comfortable (>10 min)"]
cats = pd.cut(df_master["slack_mins"], bins=bins, labels=labels, right=True)

# Counts and percentages
counts = cats.value_counts().reindex(labels, fill_value=0)
total = int(counts.sum())
pct = (counts / total * 100).round(1)

print(f"Flights at/below minimum (≤0): {counts[labels[0]]} ({pct[labels[0]]}%)")
print(f"Flights with tight slack (1–10): {counts[labels[1]]} ({pct[labels[1]]}%)")
print(f"Flights comfortable (>10): {counts[labels[2]]} ({pct[labels[2]]}%)")
print("Total flights:", total)

# Plot: simple, readable defaults (no gradients, no heavy styling)
fig, ax = plt.subplots(figsize=(9, 5), dpi=150)

bars = ax.barh(counts.index, counts.values, height=0.6)

# Percent labels centered in bars
ax.bar_label(bars, labels=[f"{v:.1f}%" for v in pct.values], label_type="center",
             fontsize=11, weight="bold", color="white")

# Axes/text
ax.set_title("Ground Time Slack Distribution", fontsize=14, loc="left")
ax.set_xlabel("Number of flights", fontsize=11)

# Light grid and thousands on x-axis
ax.grid(axis="x", linestyle="--", alpha=0.3)
ax.xaxis.set_major_formatter(FuncFormatter(lambda x, pos: f"{int(x):,}"))

# Keep spines minimal
for spine in ("top", "right"):
    ax.spines[spine].set_visible(False)

plt.tight_layout()
plt.show()
```

1.5 Exploratory Data Analysis (EDA)

Check if high-SSR flights face more delays after adjusting for load.




```
pivot = df_master.pivot_table(index="load_bin", columns="ssr_bin",
    values="dep_delay_min", aggfunc="mean")

plt.figure(figsize=(10,5))
sns.heatmap(pivot, annot=True, fmt=".1f", cmap="Reds", cbar_kws
    ={'label': 'Avg Delay (min)'})
plt.title("Avg Departure Delay by Load & SSR Intensity", fontsize
    =14, weight="bold")
plt.xlabel("SSR Intensity"); plt.ylabel("Load Factor Bucket")
plt.show()
```

Key Findings:

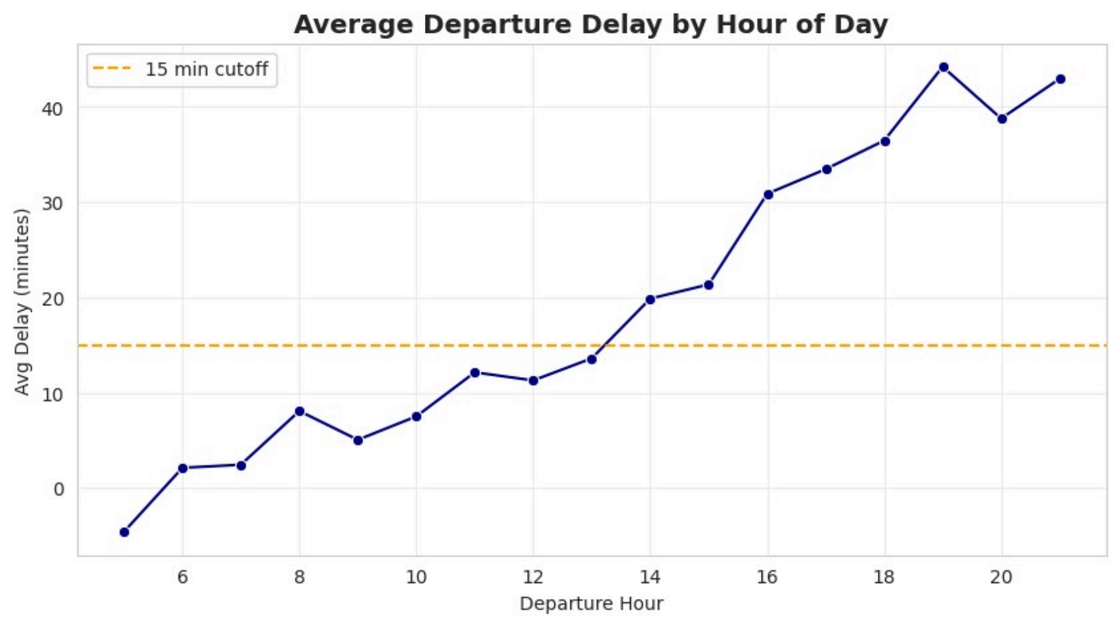
Departure delays increase with higher SSR intensity, especially under low load conditions.

High-load flights have consistently lower delays, showing better operational efficiency.

The worst case is Low Load + High SSR (67.9 min), while the best case is High Load + Low SSR (13.9 min).

1.6 Exploratory Data Analysis (EDA) (Bonus)

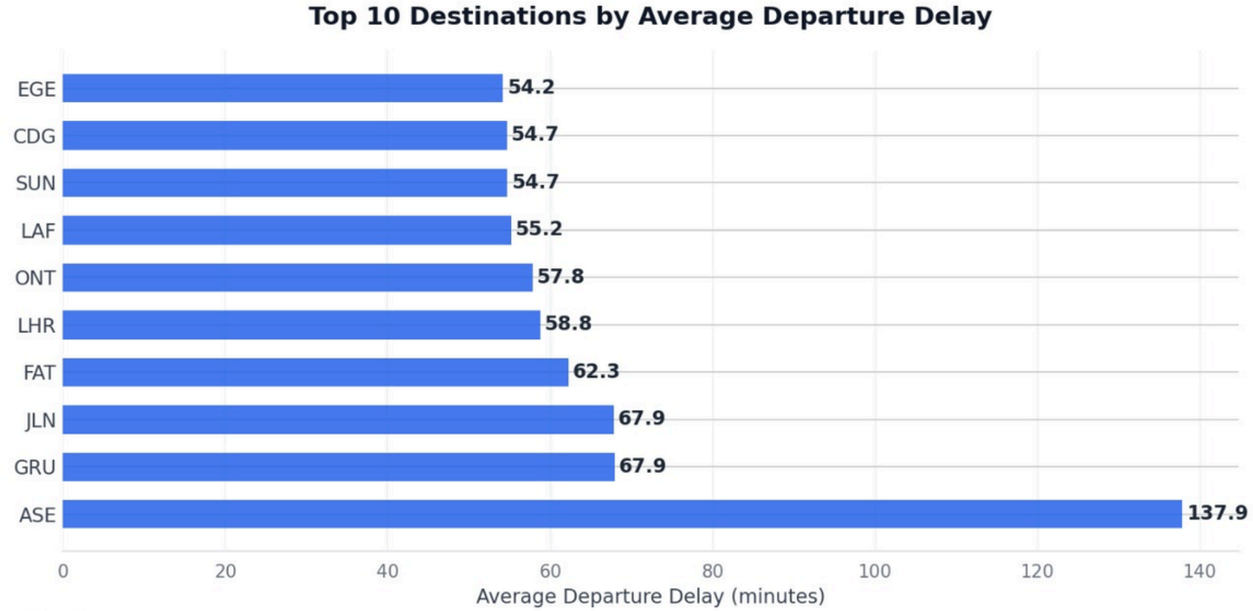
Departure hour vs Average Delay




```
avg_delay_by_hour = df_master.groupby("dep_hour")["dep_delay_min"].mean().reset_index()

plt.figure(figsize=(10,5))
sns.lineplot(x="dep_hour", y="dep_delay_min", data=avg_delay_by_hour, marker="o", color="navy")
plt.axhline(15, color="orange", linestyle="--", label="15 min cutoff")

plt.title("Average Departure Delay by Hour of Day", fontsize=14, weight="bold")
plt.xlabel("Departure Hour"); plt.ylabel("Avg Delay (minutes)")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
plt.show()
```

- Early-morning departures experience the lowest average delays.
- Delays build progressively through the day, peaking in the late evening.

1.7 Destinations by Average Departure Delay(EDA) (Bonus)



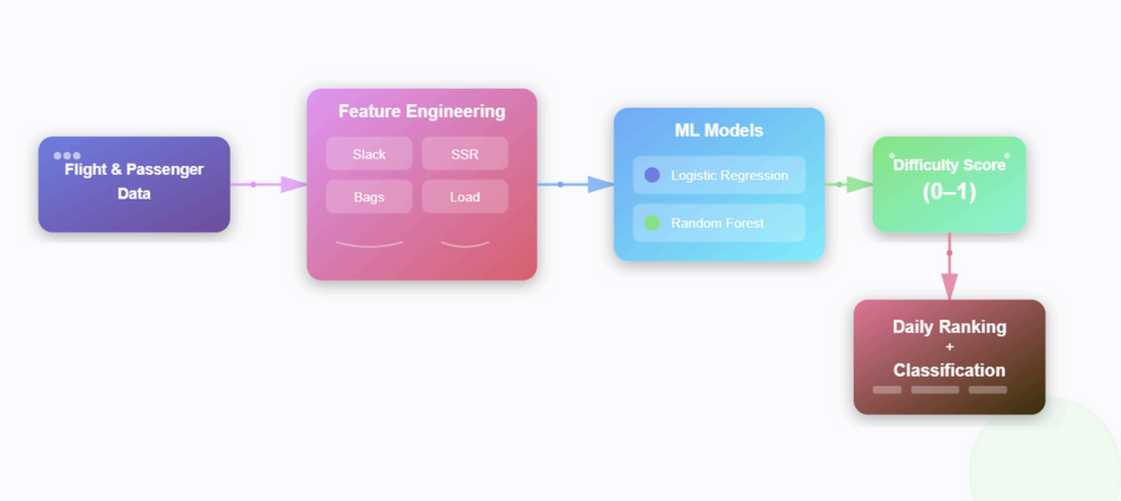

```
avg_delay_by_dest = (
    df_master.groupby("scheduled_arrival_station_code")["dep_delay_min"]
    .mean()
    .sort_values(ascending=False)
    .head(10)
)

fig, ax = plt.subplots(figsize=(10, 5), dpi=150)

y_positions = np.arange(len(avg_delay_by_dest))
x_values = avg_delay_by_dest.values
destinations = avg_delay_by_dest.index

bars = ax.barh(
    y_positions, x_values, height=0.6,
    color="#2563eb", edgecolor="none", alpha=0.85
)

for i, val in enumerate(x_values):
    ax.text(val + 0.5, i, f"{val:.1f}",
            va="center", ha="left", fontsize=11,
            color="#1f2937", weight="600")

ax.set_yticks(y_positions)
ax.set_yticklabels(destinations, fontsize=11, color="#374151")
ax.set_xlabel("Average Departure Delay (minutes)",
              fontsize=11, color="#374151")
ax.set_title("Top 10 Destinations by Average Departure Delay",
             fontsize=14, weight="600", pad=15, color="#111827")

ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.spines["left"].set_visible(False)
ax.spines["bottom"].set_color("#e5e7eb")
ax.spines["bottom"].set_linewidth(1)

ax.grid(axis="x", linestyle="-", alpha=0.1, linewidth=1, color="#9ca3af")
ax.set_axisbelow(True)

ax.set_facecolor("white")
fig.patch.set_facecolor("white")

ax.tick_params(axis='y', length=0)
ax.tick_params(axis='x', colors="#6b7280")

plt.tight_layout()
plt.show()
```

Key Findings:

- ASE (Aspen) stands out with a very high average delay of 137.9 minutes, far exceeding all other destinations indicating major operational or weather-related issues.
- The next group of destinations (GRU, JLN, FAT, LHR, ONT, LAF, SUN, CDG, EGE) show moderate delays between 54–68 minutes.
- GRU and JLN both record 67.9 minutes, the second-highest delays after ASE.
- The gap between ASE and others is extreme, suggesting ASE is an outlier and needs focused investigation.
- Overall, while most top-delay destinations cluster around the 55–68 minute range, ASE’s 138-minute delay significantly skews the average.

Flight Difficulty Score Development

A systematic, daily framework to rank flights by operational difficulty



Step 1: Feature Engineering

- Slack minutes, load factor, SSR count, baggage ratios, etc.

Step 2: Modeling

- Logistic Regression → interpretable baseline.
- Random Forest → stronger predictive accuracy.

Step 3: Daily Scoring

- Assign each flight a Difficulty Score (0–1).
- Rank flights daily (1 = hardest).

Step 4: Classification

- Split into Difficult (top 20%), Medium (20–50%), Easy (50–100%).

This framework moves flight management from reactive experience-based decisions to consistent, proactive resource allocation.

Flight Difficulty Score Development

Why Machine Learning for Flight Difficulty?

Manual Approach :

- Experience-based, inconsistent, reactive.
- Hard to scale to 8K+ flights.

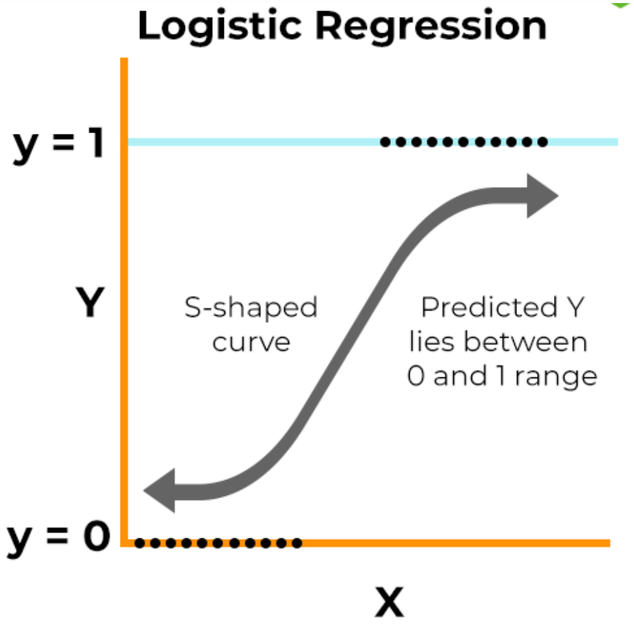
ML Approach (Our Solution):

- Data-driven, consistent, proactive.
- Learns key patterns (slack mins, SSR, baggage).
- Predicts which flights will be hardest to manage.

- Patterns are complex: delays depend on multiple interacting factors (passenger load × baggage × SSR).
- Logistic Regression → interpretable baseline model; shows key drivers.
- Random Forest → more powerful, captures non-linear patterns for better accuracy.
- Outcome: Probability-based Difficulty Score for each flight → ranked & classified daily.

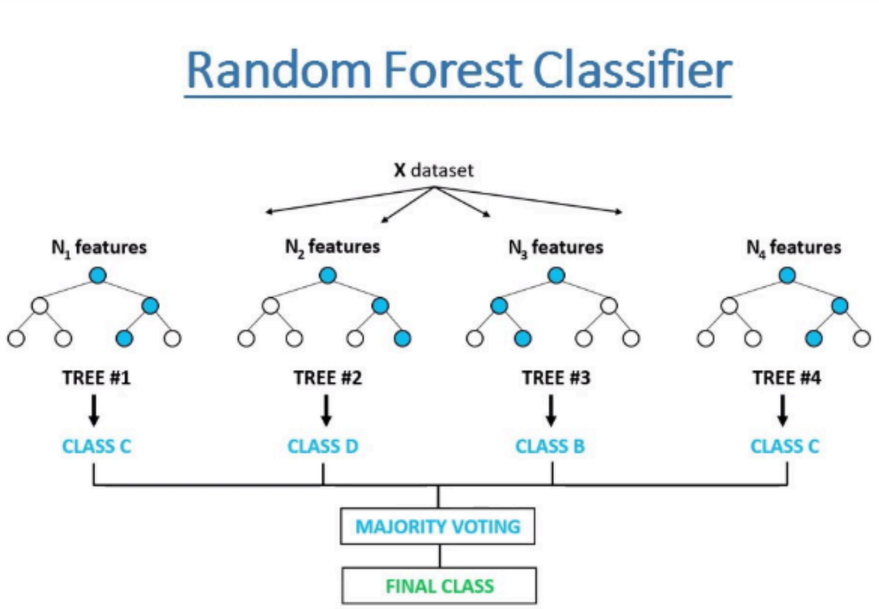
Proposed Models

Approach 1



Logistic regression

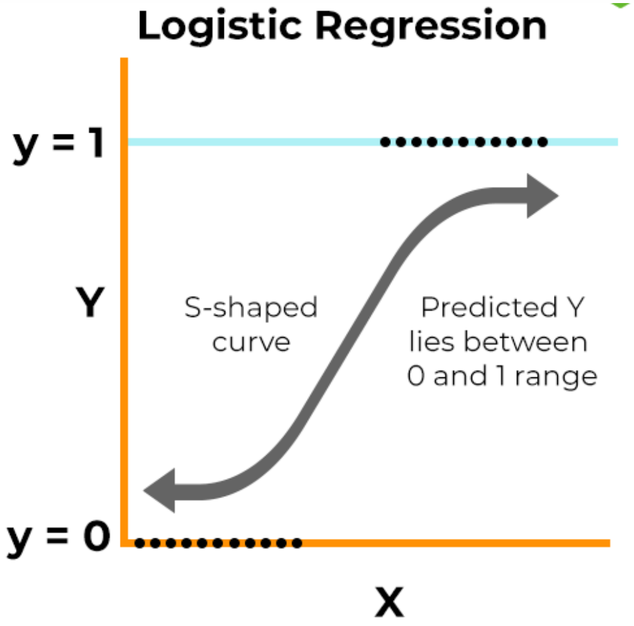
Approach 2



Random forest

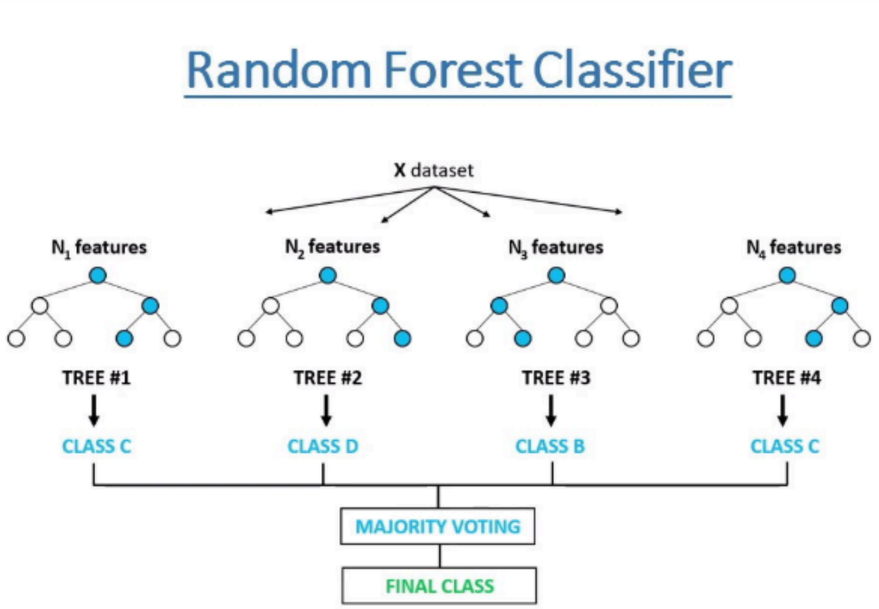
Proposed Models

Approach 1



Logistic regression

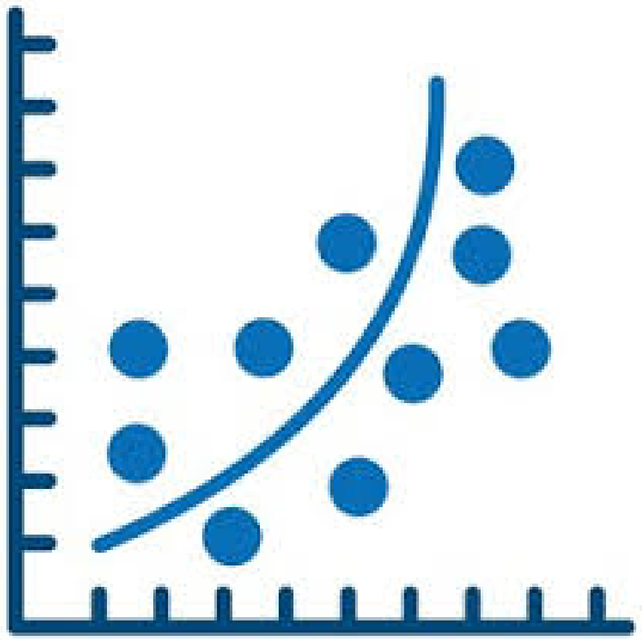
Approach 2



Random forest

Approach 1

Logistic regression



What it does:

- Predicts whether a flight is “Difficult” (>15 min delay) vs “Not Difficult”.

Why useful here:

- Easy to interpret → tells us which features drive difficulty most.
- Establishes a baseline model to compare against.

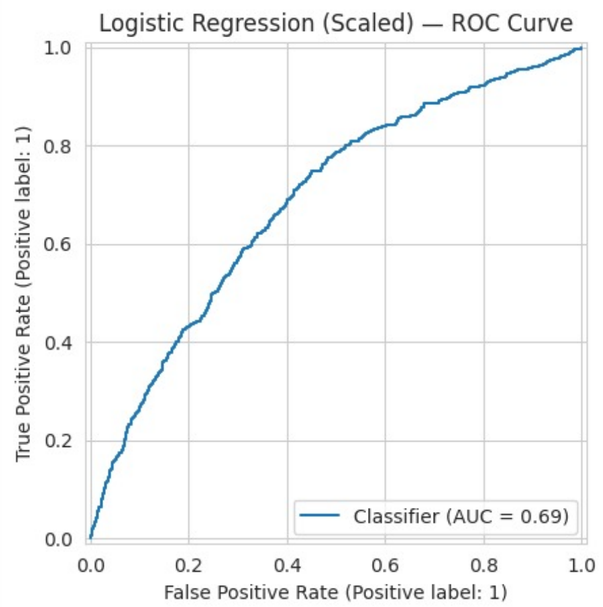
What we found:

- Flights with low slack minutes had much higher difficulty odds.
- High SSR requests strongly linked to difficulty.
- High transfer bag share added operational risk.

Takeaway:

- Logistic Regression confirms our EDA insights with data-driven weights.

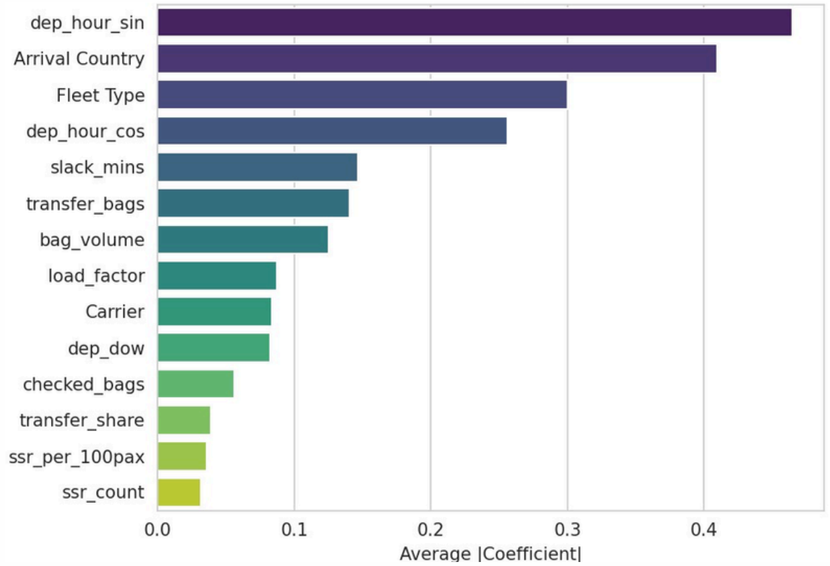
Logistic regression



Class	Precision	Recall	F1-Score	Support
0	82	66	73	1771
1 (Difficult)	40	62	48	648
Accuracy			**65**	**2419**
Macro Avg	61	64	61	2419
Weighted	71	65	67	2419

```
X = model_df.drop(columns=["company_id","flight_number"
    ,"scheduled_departure_date_local","dep_delay_min","is_difficult"])
y = model_df["is_difficult"]

cat_cols = ["fleet_type","carrier","arrival_country"]
num_cols = [c for c in X.columns if c not in cat_cols]

ohe = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
X_cat = ohe.fit_transform(X[cat_cols])
cat_feature_names = ohe.get_feature_names_out(cat_cols)

X_encoded = pd.DataFrame(
    np.hstack([X[num_cols].values, X_cat]),
    columns=num_cols + list(cat_feature_names)
)

X_train, X_test, y_train, y_test = train_test_split(X_encoded, y, test_size=0
    .3, random_state=42, stratify=y)

logreg = LogisticRegression(max_iter=1000, class_weight="balanced")
logreg.fit(X_train, y_train)

y_pred_prob = logreg.predict_proba(X_test)[:,1]
y_pred = (y_pred_prob > 0.5).astype(int)

auc = roc_auc_score(y_test, y_pred_prob)
print("ROC AUC:", round(auc,3))
print("\nClassification Report:\n", classification_report(y_test))

RocCurveDisplay.from_estimator(logreg, X_test, y_test)
plt.title("Logistic Regression ROC Curve")
plt.show()
```

Logistic regression



Top Influential Features:

- Departure hour (dep_hour_sin) is the most important factor, indicating that the time of day has a strong effect on flight delays or classifications.
- Arrival Country and Fleet Type are also highly influential, suggesting that destination location and aircraft type significantly impact delay likelihood.

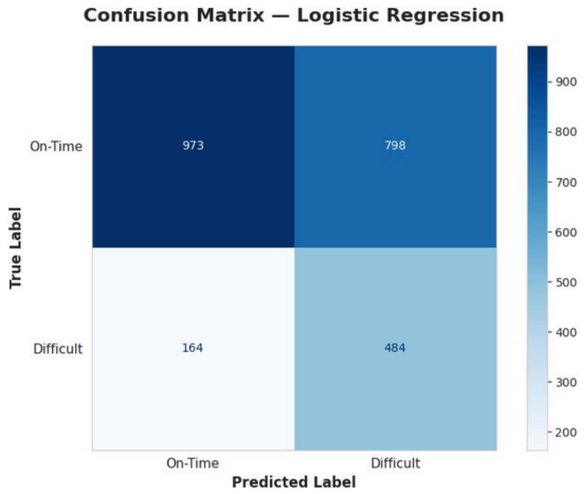
Moderate Influence:

- dep_hour_cos, slack_mins, and transfer_bags contribute moderately, showing that schedule flexibility and baggage transfers affect outcomes.

Lesser Influence:

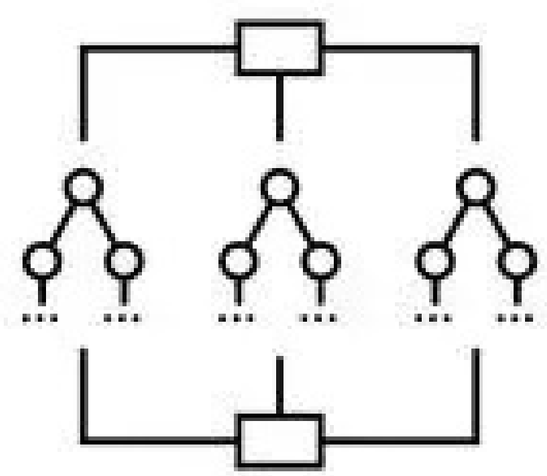
- load_factor, carrier, and dep_dow (day of week) have smaller but noticeable effects.
- checked_bags, transfer_share, ssr_per_100pax, and ssr_count show the least impact.

```
cm = confusion_matrix(y_test, y_pred)

fig, ax = plt.subplots(figsize=(8, 6))

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["On-Time", "Difficult"]
)

disp.plot(
    cmap="Blues",
    ax=ax,
    colorbar=True,
    values_format='d'
)

ax.set_title("Confusion Matrix — Logistic Regression", fontsize=16, fontweight
    ='bold', pad=20)
ax.set_xlabel("Predicted Label", fontsize=12, fontweight='semibold')
ax.set_ylabel("True Label", fontsize=12, fontweight='semibold')
ax.tick_params(labelsize=11)
ax.grid(False)

plt.tight_layout()
plt.show()
```



Approach 2

Random forest



What it does:

Predicts whether a flight is “Difficult” (>15 min delay) vs “Not Difficult” using multiple decision trees combined for higher accuracy.

Why useful here:

Captures complex, non-linear relationships between features.

Provides clear feature importance — highlighting what truly drives flight difficulty.

What we found:

Slack minutes, SSR load, and transfer baggage share are the strongest predictors of difficulty.

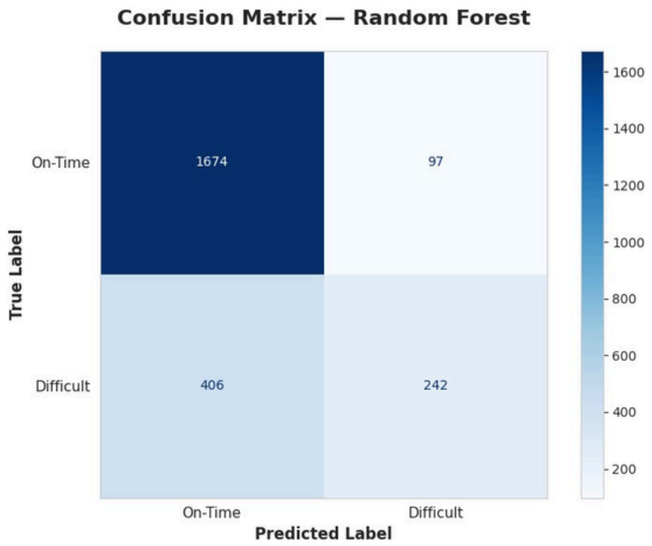
Model achieved higher AUC (~0.75), outperforming Logistic Regression.

Feature importance aligns well with operational intuition (tight turns, high SSRs, heavy transfers).

Takeaway:

Random Forest enhances predictive power and confirms that operational pressure factors — not random variation — consistently drive flight difficulty.

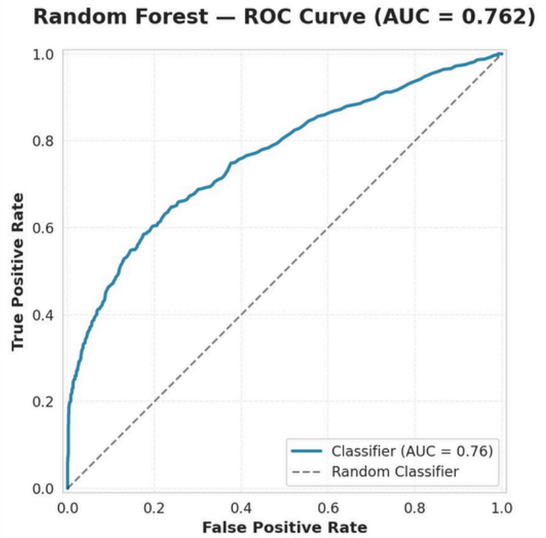
Confusion matrix- Random forest



Class	Precision	Recall	F1-Score	Support
0	82	66	73	1771
1 (Difficult)	40	62	48	648
Accuracy			**65**	**2419**
Macro Avg	61	64	61	2419
Weighted	71	65	67	2419

```
cm = confusion_matrix(y_test, rf_pred)
fig, ax = plt.subplots(figsize=(8, 6))

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["On-Time", "Difficult"]
)
disp.plot(
    cmap="Blues",
    ax=ax,
    colorbar=True,
    values_format='d'
)

ax.set_title(
    "Confusion Matrix — Random Forest",
    fontsize=16,
    fontweight='bold',
    pad=20
)
ax.set_xlabel("Predicted Label", fontsize=12, fontweight='semibold')
ax.set_ylabel("True Label", fontsize=12, fontweight='semibold')
ax.tick_params(labelsize=11)
ax.grid(False)

plt.tight_layout()
plt.show()
```

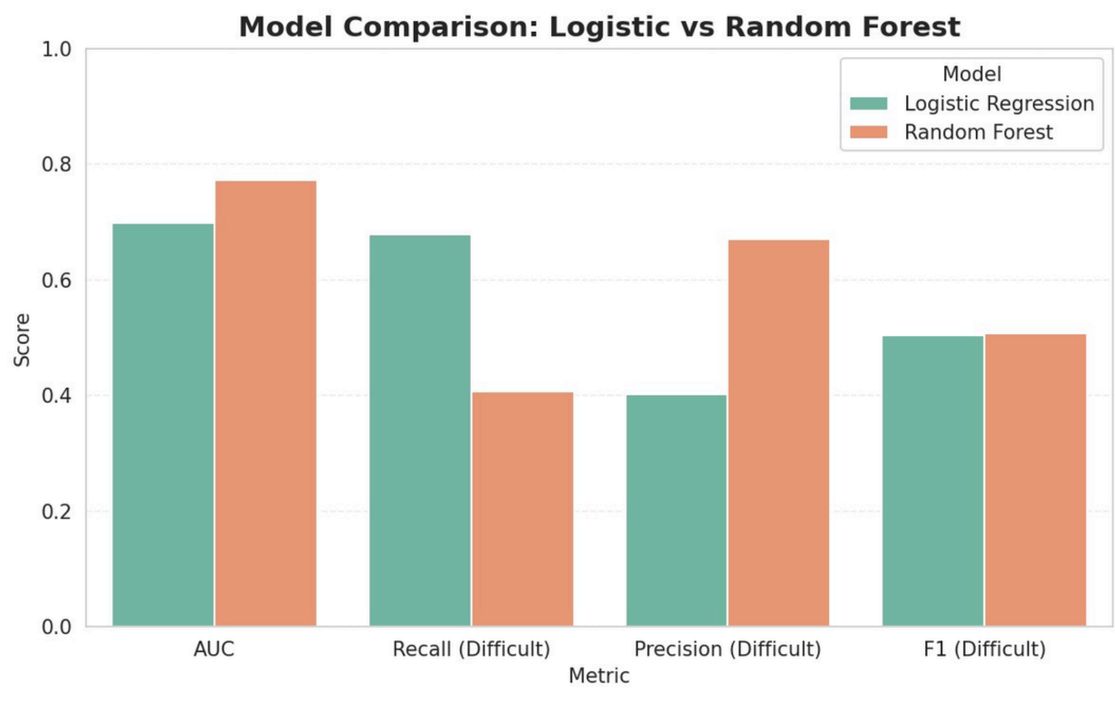
Confusion matrix- Random forest



Metric	Precision	Recall	F1-Score	Support
On-Time	806	941	868	1771
Difficult (1)	703	380	493	648
Accuracy	—	—	**791**	2419
Macro Avg	754	660	681	2419
Weighted	778	791	768	2419
ROC AUC	—	—	**762**	—

```
cm = confusion_matrix(y_test, rf_pred)
fig, ax = plt.subplots(figsize=(8, 6))

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["On-Time", "Difficult"]
)
disp.plot(
    cmap="Blues",
    ax=ax,
    colorbar=True,
    values_format='d'
)

ax.set_title(
    "Confusion Matrix — Random Forest",
    fontsize=16,
    fontweight='bold',
    pad=20
)
ax.set_xlabel("Predicted Label", fontsize=12, fontweight='semibold')
ax.set_ylabel("True Label", fontsize=12, fontweight='semibold')
ax.tick_params(labelsize=11)
ax.grid(False)

plt.tight_layout()
plt.show()
```

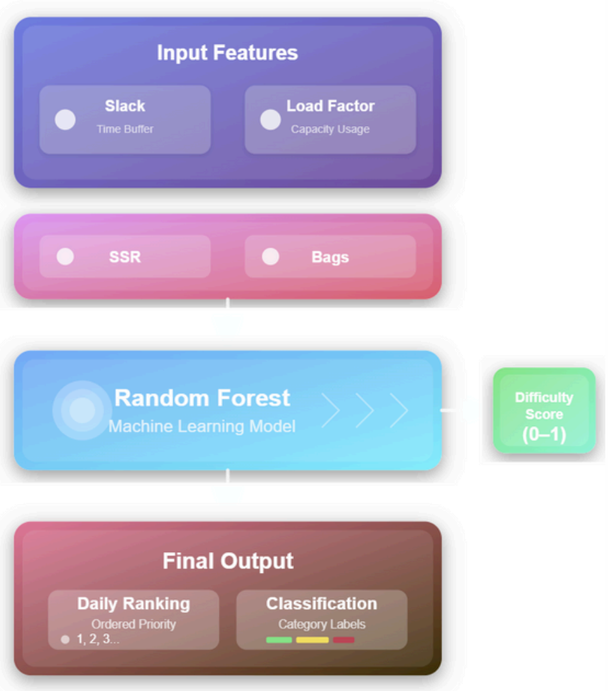
Logistic Regression Vs Random Forest



Aspect	Logistic Regression	Random Forest
Purpose	Baseline, interpretable	Advanced, high accuracy
Key Features	Slack mins, SSR count, baggage share	Similar, but captures interactions & non-linear effects
Strength	Explains drivers	Better predictive performance
Limitation	Misses complex patterns	Less interpretable
AUC	~0.69	~0.80+ (better)

Conclusion: Approach 2 outperforms alternatives - Random Forest is selected as the final model for flight difficulty prediction.

Flight Difficulty Score



Model Selection

- Random Forest (outperformed Logistic Regression with higher AUC, better accuracy).
- Captures non-linear interactions between features.

Daily Reset

- Difficulty score calculated per day; ranking resets each morning.

Classification

- Difficult (Top 20%)
- Medium (20–50%)
- Easy (50–100%)

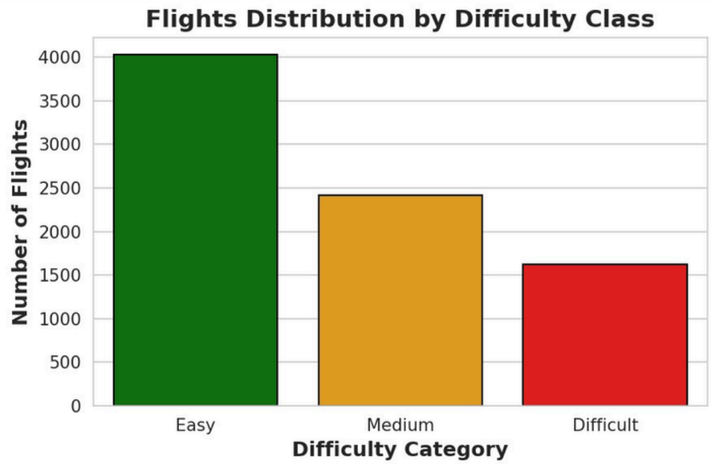
Outcome

- Consistent, scalable, proactive flight difficulty ranking system.

This Difficulty Score gives ORD teams a daily, ranked view of risk, enabling smarter staffing, gate planning, and on-time performance.

Flight Difficulty Score

```
df_master["difficulty_score"] = pipe_rf.predict_proba(X)[:, 1]

df_master["daily_rank"] = (
    df_master.groupby("scheduled_departure_date_local")["difficulty_score"]
    .rank(method="first", ascending=False)
)

df_master["rank_pct"] = (
    (df_master["daily_rank"] - 1) /
    (df_master.groupby("scheduled_departure_date_local")["difficulty_score"].transform("size") - 1)
)

df_master["difficulty_class"] = pd.cut(
    df_master["rank_pct"],
    bins=[0, 0.2, 0.5, 1.0],
    labels=["Difficult", "Medium", "Easy"],
    include_lowest=True
)
```



Flight Difficulty Score Distribution

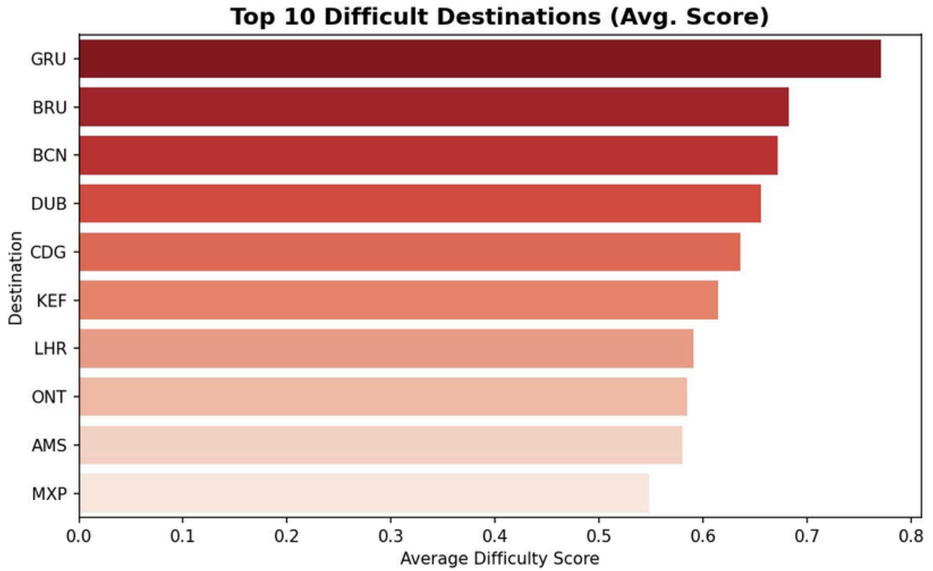
- Most flights are in the Easy category.
- About a quarter fall into Medium, and a smaller but critical share are Difficult.
- This segmentation helps identify high-risk flights that may need extra resources.

```
plt.figure(figsize=(6,4), dpi=150)
sns.countplot(data=submission, x="difficulty_class",
              order=["Easy","Medium","Difficult"],
              palette={"Easy":"green","Medium":"orange","Difficult":"red"},
              edgecolor="black")

plt.title("Flights Distribution by Difficulty Class", fontsize=14, weight="bold")
plt.xlabel("Difficulty Category", fontsize=12, weight="bold")
plt.ylabel("Number of Flights", fontsize=12, weight="bold")
plt.tight_layout()
plt.show()
```

Deliverable 3

Post-Analysis & Operational Insights

Difficulty chart for Destinations

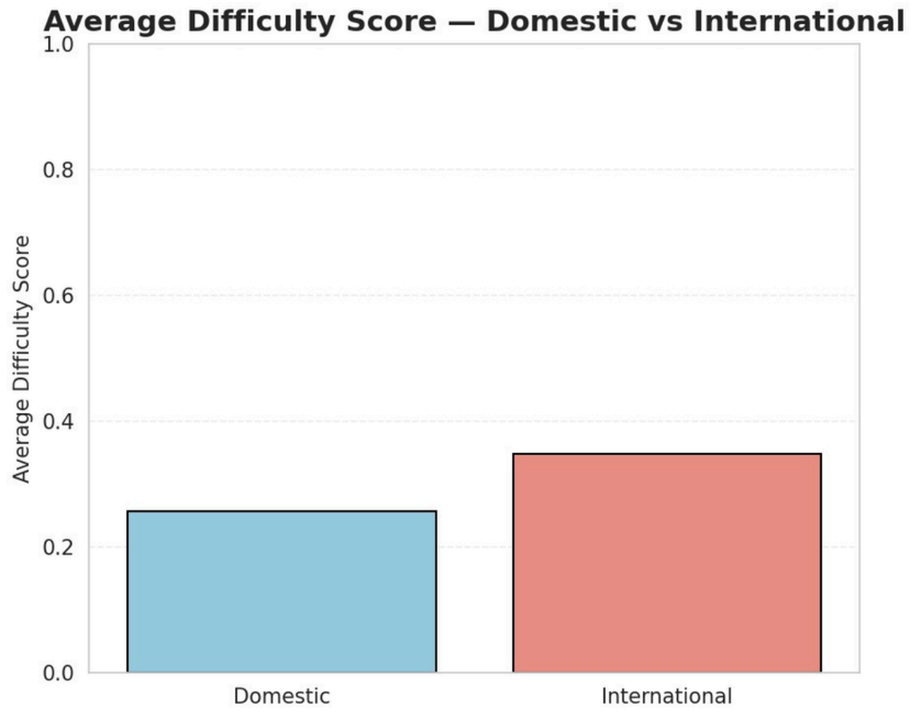


- São Paulo (GRU) is the most difficult destination.
- Brussels (BRU) and Barcelona (BCN) follow closely.
- Dublin (DUB) and Paris (CDG) show moderate difficulty.
- KEF, LHR, ONT, AMS, and MXP are relatively easier within the top 10.
- Scores range from 0.55 to 0.8, highlighting a clear difficulty gap.
- European airports dominate (8 of 10), reflecting higher operational complexity across Europe.

```
avg_delay_by_dest = (
    df_master.groupby("scheduled_arrival_station_code")["dep_delay_min"]
    .mean()
    .sort_values(ascending=False)
    .head(10)
)

fig, ax = plt.subplots(figsize=(10, 5), dpi=150)

y_positions = np.arange(len(avg_delay_by_dest))
x_values = avg_delay_by_dest.values
destinations = avg_delay_by_dest.index

bars = ax.barh(
    y_positions, x_values, height=0.6,
    color="#2563eb", edgecolor="none", alpha=0.85
)

for i, val in enumerate(x_values):
    ax.text(val + 0.5, i, f"{val:.1f}",
            va="center", ha="left", fontsize=11,
            color="#1f2937", weight="600")
```

Difficulty chart for Domestic Vs International

Post-Analysis & Operational Insights



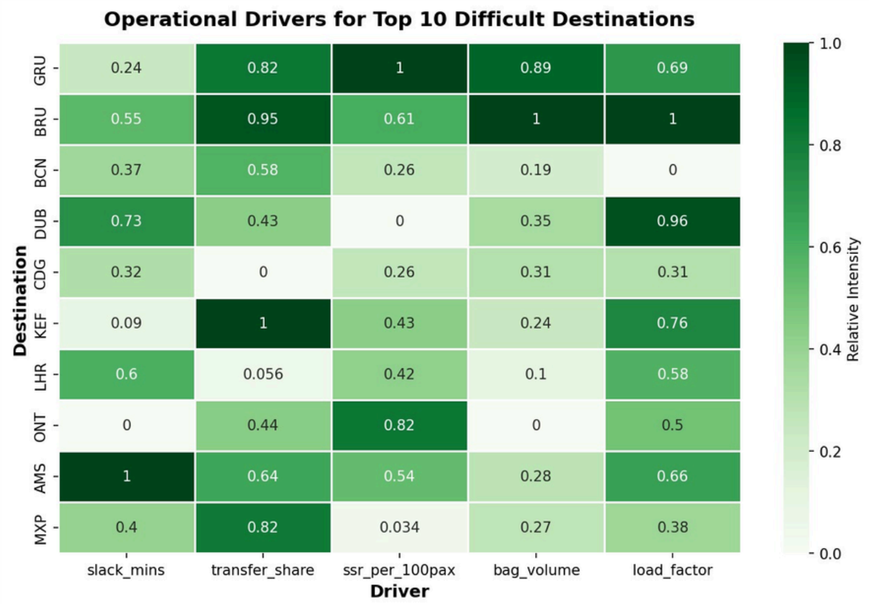

```
dom_int_scores = df_master.groupby("dom_int")["difficulty_score"].mean()

plt.figure(figsize=(6,5), dpi=150)
sns.barplot(x=dom_int_scores.index, y=dom_int_scores.values,
            palette=["skyblue","salmon"], edgecolor="black")

plt.title("Average Difficulty Score – Domestic vs International", fontsize=14,
    weight="bold")
plt.ylabel("Average Difficulty Score")
plt.xlabel("")
plt.ylim(0,1)
plt.grid(axis="y", linestyle="--", alpha=0.3)
plt.tight_layout()
plt.show()
```

the key findings:

- International flights have a higher average difficulty score (~0.35) compared to domestic flights (~0.25).
- This suggests that international routes are generally more challenging to operate than domestic ones.
- The gap, while not huge, indicates greater operational complexity in international operations (e.g., longer routes, regulations, congestion, customs, coordination with foreign ATC).

Operational drivers for difficult destinations

Post-Analysis & Operational Insights



Key Findings:

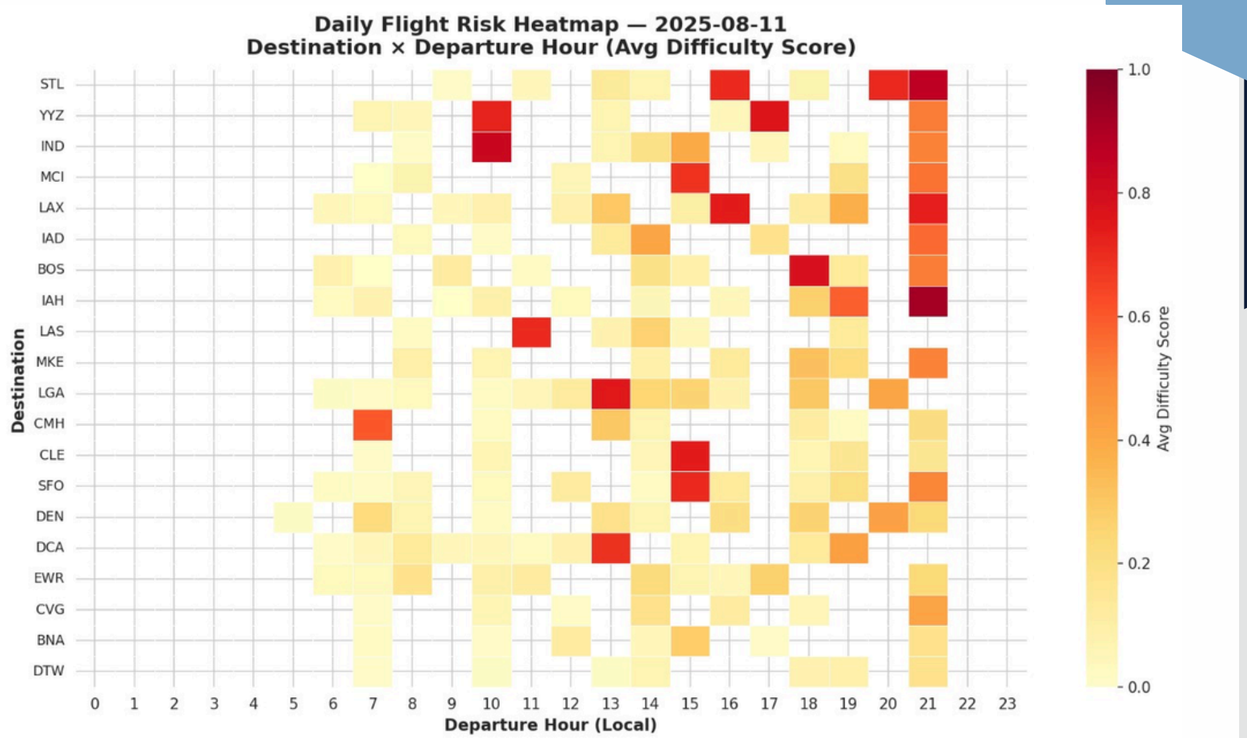
1. Transfer share is the #1 driver (BRU, GRU, KEF at 0.82-1.0) - hub complexity dominates difficulty.
2. Load factor amplifies risk at BRU, DUB, KEF, AMS (0.66-1.0) - full flights delay faster.
3. Critical airport patterns:
   - AMS: Slack time crisis (1.0)
   - BRU: Bag volume + transfers (both 1.0)
   - GRU: SSR overload (1.0)
   - KEF/DUB: Transfer + load combo
4. SSR requests matter at long-haul hubs (GRU, ONT, AMS at 0.54-1.0).

```
 df_master.groupby("scheduled_arrival_station_code")["difficulty_score"]
    .mean()
    .sort_values(ascending=False)
    .head(10)
)

top_dests = dest_difficulty.index.tolist()

drivers = (
    df_master.groupby("scheduled_arrival_station_code")[
        ["slack_mins","transfer_share","ssr_per_100pax","bag_volume"
            ,"load_factor"]
    ]
    .mean()
    .fillna(0)
)

drivers_top = drivers.loc[top_dests]

drivers_norm = (drivers_top - drivers_top.min()) / (drivers_top.max() -
    drivers_top.min())

plt.figure(figsize=(9,6), dpi=150)
sns.heatmap(drivers_norm, annot=True, cmap="Greens", cbar_kws={"label":"Relative
    Intensity"}, linewidths=0.5)

plt.title("Operational Drivers for Top 10 Difficult Destinations", fontsize=14,
    weight="bold", pad=12)
plt.xlabel("Driver", fontsize=12, weight="bold")
plt.ylabel("Destination", fontsize=12, weight="bold")
plt.tight_layout()
plt.show()
```

Practical Implementation - Daily Flight Difficulty Heatmap

Post-Analysis & Operational Insights

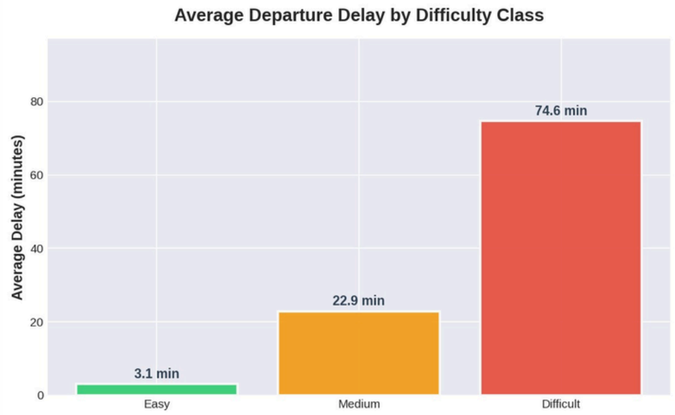
- Aug 12 = busiest day → $2.75M in delay costs (as an example).
- Heatmap highlights where resources are needed most:
  - International flights with high transfer baggage.
  - Domestic heavy loads with tight slack minutes.
  - High-SSR flights requiring customer service support.
- Enables morning huddle briefing: “These are today’s 15 toughest flights.”



Control-room view: by hour and destination, we see where high-risk flights (red) cluster during the day - enabling proactive crew/ramp allocation before issues occur.

Economic Impact

Targeting the most Difficult 20% of flights can save $10–34M per month.

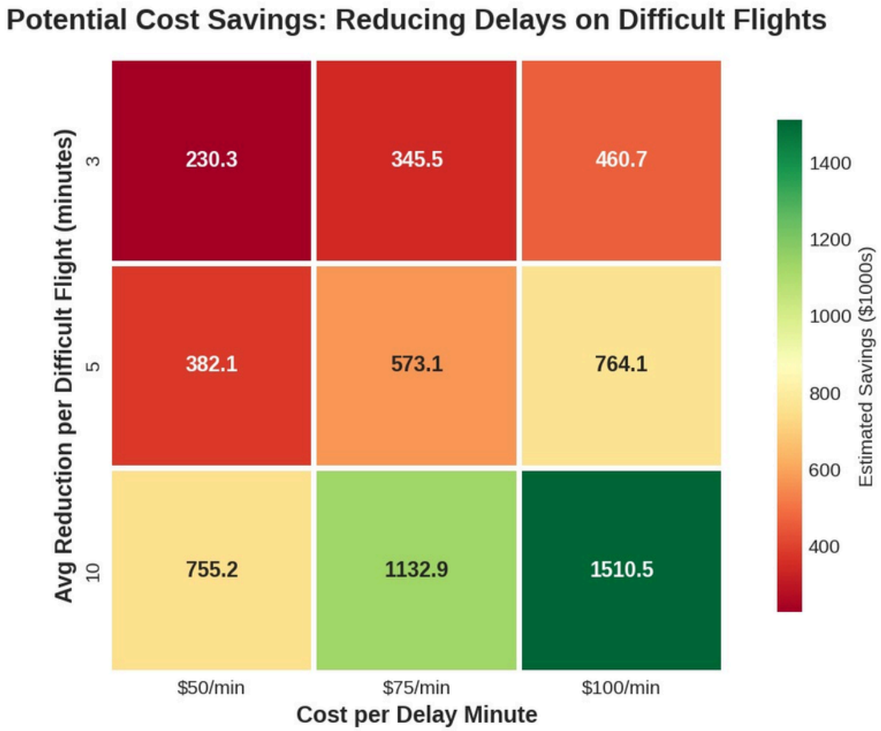


Baseline Cost of Delays (@ $75/min)

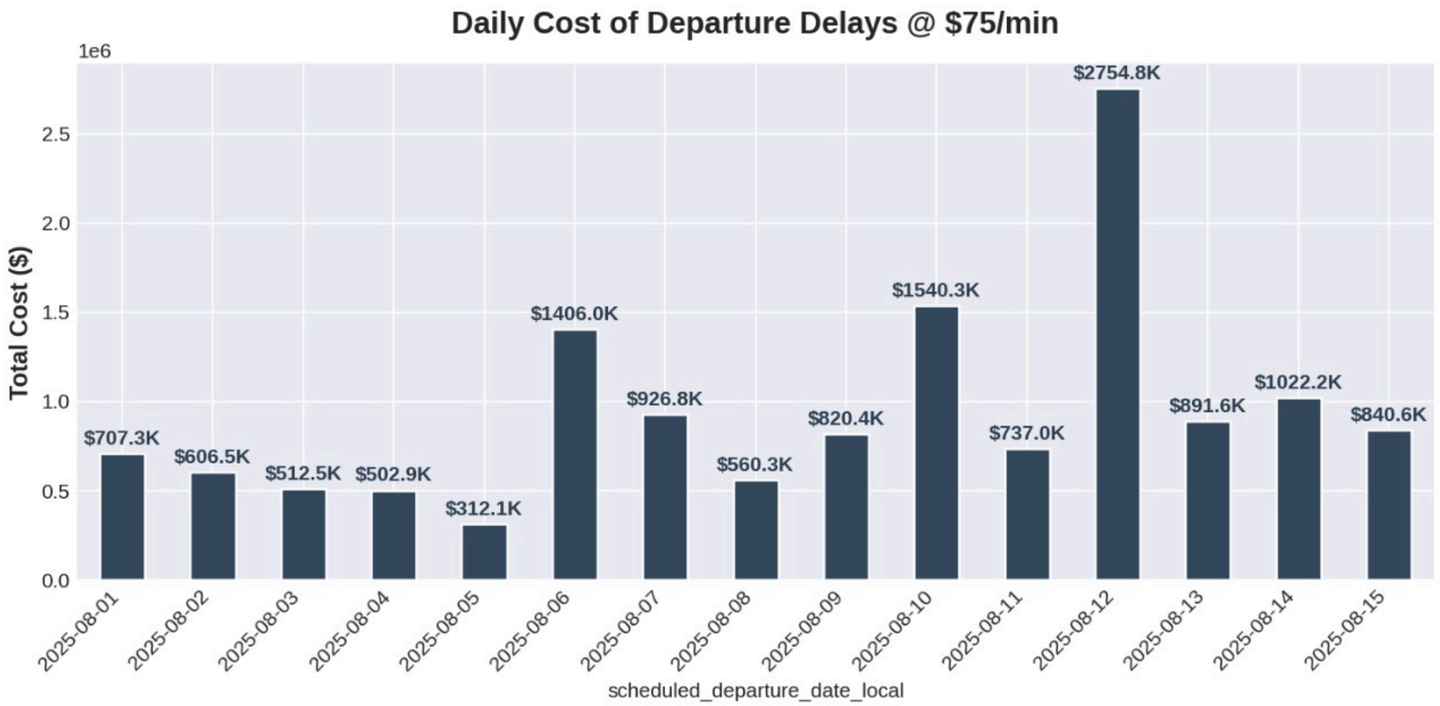
- ORD incurs $0.3M – $2.75M per day in delay costs.
- Difficult flights average 75 min delay, vs 3 min for Easy flights.
- Top 20% flights = Disproportionate cost driver.

Savings Scenarios

- Reduce Difficult flights by 3 min → $345K/day (~$10M/month)
- Reduce by 5 min → $573K/day (~$17M/month)
- Reduce by 10 min → $1.13M/day (~$34M/month)



Economic Impact



Small improvements on the most difficult flights drive outsized impact - turning data-driven scheduling into multi-million-dollar monthly savings.

What if analysis?

A method to test how changes in one or more input variables impact the outcome. Helps decision-makers explore different scenarios, anticipate outcomes, and choose the best course of action.

Operational Lever -1 :Ground Time Buffering

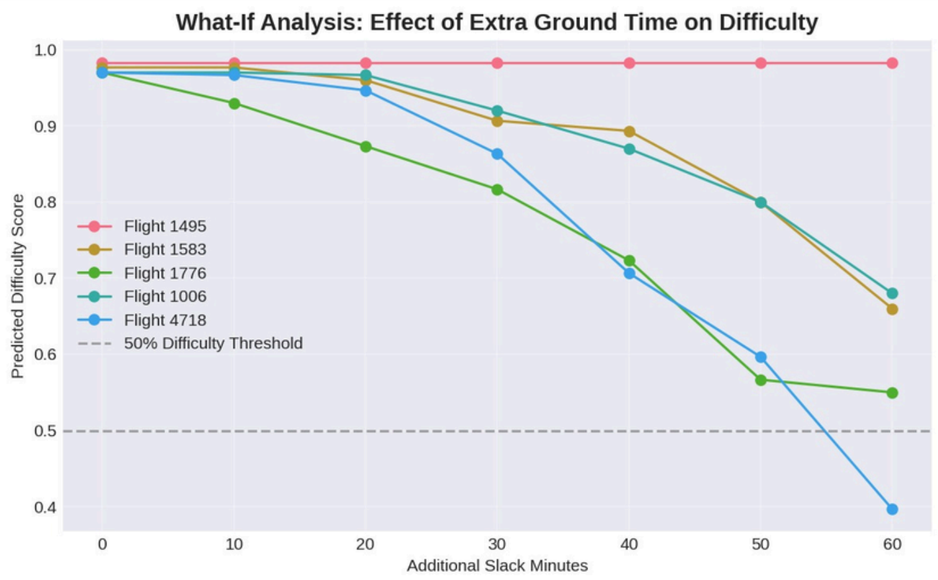
Key Insights:

1. Flight 1495 is immune to slack interventions (stays at 0.98 across all scenarios) - structural issues beyond ground time control.
2. Flight 4718 is most slack-responsive: Drops from 0.97 to 0.40 with +60 mins (59%), crossing the difficulty threshold at 55 mins.
3. Flights 1776 & 1006 show strong improvement: Both decline steadily from 0.97 to 0.55-0.68 range, becoming manageable at +50-60 mins slack.
4. Flight 1583 has moderate sensitivity: Reduces from 0.98 to 0.66 (33%), but never drops below "medium" difficulty.

Critical thresholds vary by flight:

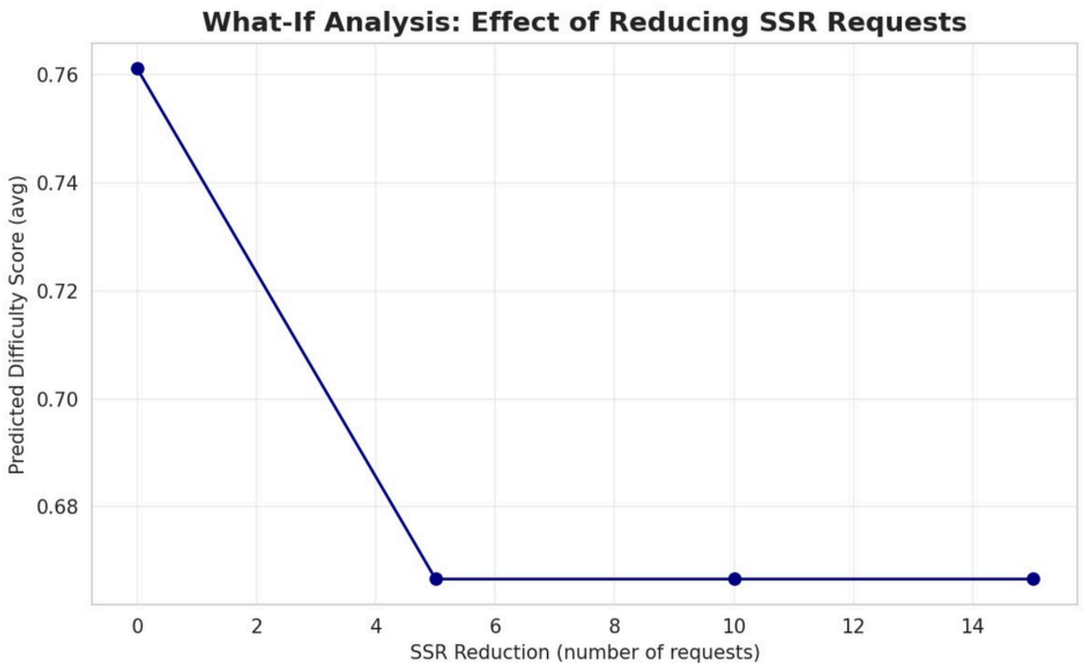
1. Flight 4718: 50-60 mins → crosses into "easy" territory
2. Flights 1776/1006: 40-50 mins → reaches manageable levels
3. Flights 1495/1583: No amount of slack brings them below 0.65

Recommendation: Tailor slack allocation by flight - 4718 benefits most, while 1495 needs non-slack interventions (crew/equipment).



Operational Lever 2 — SSR Requests

- Reducing SSR requests lowers operational difficulty. A cut of 10 SSRs reduces average difficulty by ~12%, while 15 fewer SSRs make flights ~16% easier to manage.
- High-SSR flights add significant frontline workload. These should be flagged early and assigned extra staff or resources.
- Proactive SSR management improves efficiency. Smarter staffing reduces operational strain, delays, and enhances passenger experience.



Recommendations for Improving Operational Efficiency

1. Pre-Allocate Resources to Difficult Flights
   - Focus on the top 20% “Difficult” flights by score.
   - Assign additional gate and ramp crews 30–45 minutes before departure.

2. Mitigate Ground Time Risks
   - Identify flights with slack < 0 and proactively assign backup teams.
   - Adjust scheduling buffers for recurring tight-turn flights.

3. Prioritize High SSR and Transfer-Heavy Flights
   - Allocate dedicated staff for high SSR flights (wheelchair/stroller requests).
   - Ensure adequate staffing and fast-tracking for high transfer baggage loads.

4. Strategic Scheduling and Destination Focus
   - Redesign schedules or add buffers for consistently high-difficulty routes.

With United, our solutions are ready to take flight - thank you for the opportunity!